数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学

大数据与互联网学院

第3章 关系数据库存储管理

- 3.1 SQL概述
- 3.2 数据定义
- 3.3 基本SQL查询
- 3.4 数据更新
- 3.5 视图
- 3.6 小结

3.5 视图

- 视图的特点
  - 虚表，是从一个或几个基本表（或视图）导出的表
  - 只存放视图的定义，不存放视图对应的数据
  - 基表中的数据发生变化，从视图中查询出的数据也随之改变

建立视图

- 关系数据库管理系统执行**CREATE VIEW**语句时只是把视图定义存入数据字典，并不执行其中的SELECT语句。
- 在对视图查询时，按视图的定义从基本表中将数据查出。

3.5 视图

- 3.5.1 定义视图
- 3.5.2 查询视图
- 3.5.3 更新视图

3.5.1 定义视图

定义视图以CREATE VIEW 开始，加上视图名称，后面可以是选择的属性列表。若没有指定属性列表，则视图包含的属性就是之后的查询结果。

```
<view definition> ::=
CREATE VIEW <table name> [ <left paren> <view column list> <right paren> ]
AS <query expression>
[ WITH [ <levels clause> ] CHECK OPTION ]
<view column list> ::= <column name list>
```

之后由AS关键字引出一个查询表达式。最后如果指定 WITH CHECK OPTION，则视图应该是可以更新的。

3.5.1 定义视图

通过该视图插入或更新后的元组需要满足视图定义。

```
<levels clause> ::= CASCADED | LOCAL
```

默认是CASCADED，即该视图和其所依赖的视图都应满足此条件。

若指定LOCAL，则只检查当前视图。

Mysql定义视图语法

```
CREATE
    [OR REPLACE]
    [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
    [DEFINER = user]
    [SQL SECURITY { DEFINER | INVOKER }] //缺省INVOKER
    VIEW view_name [(column_list)]
    AS select_statement
    [WITH [CASCADED | LOCAL] CHECK OPTION]
```

MERGE：缺省，视图可更新

TEMPTABLE：不可更新

定义视图（续）

- WITH CHECK OPTION
  - 对视图进行UPDATE，INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件（即子查询中的条件表达式）
- 子查询可以是任意的SELECT语句，是否可以含有ORDER BY子句和DISTINCT短语，则决定具体系统的实现。

定义视图（续）

- 组成视图的属性列名：全部省略或全部指定
  - 全部省略:
    - 由子查询中SELECT目标列中的诸字段组成
  - 明确指定视图的所有列名:
    - 某个目标列是聚集函数或列表达式
    - 多表连接时选出了几个同名列作为视图的字段
    - 需要在视图中为某个列启用新的更合适的名字

3.5.1 定义视图

[例3.23]定义一个视图，存放订单的总价。

没有指定属性列表，因此其包含后面查询的两个列，OID和SUM。

```
CREATE VIEW view_TotalPrice
        AS
        SELECT OID, SUM(Quantity * Price * Discount) AS TotalPrice
        FROM Products JOIN OrderItems ON Products.PID = OrderItems.PID
        GROUP BY OID;
```

该视图包含了分组聚集的结果，因此不能更新。

3.5.1 定义视图

[例3.24]定义一个视图，里面只存放所有类别是“数码产品”的商品信息。

```
CREATE VIEW view_DigitalProducts
    AS
    SELECT *
    FROM Products
    WHERE Category = '数码产品'
    WITH CHECK OPTION;
```

没有指定属性列表，因此其包含后面查询的所有列。

该视图可以更新，也指定了WITH CHECK OPTION。

对该视图进行修改和插入时仍需保证该视图只有数码产品，条件：Category = '数码产品'

定义视图（续）

- 基于多个基表的视图

[例,第6版] 建立信息系选修了1号课程的学生的视图（包括学号、姓名、成绩）。

```
CREATE VIEW IS_S1(Sno,Sname,Grade)
 AS
 SELECT Student.Sno,Sname,Grade
 FROM  Student,SC
 WHERE  Sdept= 'IS' AND
              Student.Sno=SC.Sno AND
              SC.Cno= '1';
```

定义视图（续）

- 基于视图的视图

[例,第6版] 建立信息系选修了1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_S2
AS
SELECT Sno,Sname,Grade
FROM  IS_S1
WHERE  Grade>=90;
```

定义视图（续）

- 带表达式的视图

[例,第6版] 定义一个反映学生出生年份的视图。

```
CREATE  VIEW BT_S(Sno,Sname,Sbirth)
 AS
 SELECT Sno,Sname,2024-Sage
 FROM  Student;
```

定义视图（续）

- 分组视图

[例,第6版] 将学生的学号及平均成绩定义为一个视图

```
CREAT  VIEW S_G(Sno,Gavg)
    AS
    SELECT Sno,AVG(Grade)
    FROM  SC
    GROUP BY Sno;
```

定义视图（续）

[例,第6版]将Student表中所有女生记录定义为一个视图

```
CREATE VIEW F_Student(F_Sno,name,sex,age,dept)
 AS
 SELECT  *                    /*不指定属性列*/
 FROM  Student
 WHERE Ssex='女';
```

缺点:

如修改基表Student的结构，Student表与F_Student视图的映象关系被破坏，导致该视图不能正确工作。

3.5 视图

- 3.5.1 定义视图
- 3.5.2 查询视图
- 3.5.3 更新视图

- 用户角度：查询视图与查询基本表相同
- 关系数据库管理系统实现视图查询的方法
  - 视图消解法（View Resolution）
    - 进行有效性检查
    - 转换成等价的对基本表的查询
    - 执行修正后的查询

3.5.2 查询视图

查询视图与查询基础表的语法相同。

[例3.25]根据已定义的视图view_TotalPrice，查询编号为'O001'的订单总价。

```
SELECT *
        FROM view_TotalPrice
        WHERE OID = 'O001’
```

查询结果：

OID	总价
O001	3148.15

查询视图（续）

[例,第6版] 在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT   Sno,Sage
FROM     IS_Student
WHERE    Sage<20;
```

DBMS在视图消解转换后的查询语句转化为：

```
SELECT  Sno,Sage
FROM  Student
WHERE  Sdept= 'IS'  AND  Sage<20;
```

查询视图表结构

DESCRIBE|DESC 视图名;

3.5 视图

- 3.5.1 定义视图
- 3.5.2 查询视图
- 3.5.3 更新视图

3.5.3 更新视图

更新视图与更新基础表语法相同。

[例3.26]根据视图view_ DigitalProducts进行更新，修改ID为P0001的商品，将其价格降价100元，更新后查询视图view_ DigitalProducts。

```
UPDATE view_DigitalProducts
        SET Price = Price - 100
        WHERE PID    ='P0001';
```

查询结果：

PID	PName	Price	Category	SID
P0001	智能手机	1899	数码产品	S001

更新视图（续）

[例,第6版] 向信息系学生视图IS_S中插入一个新的学生记录，其中学号为”201215129”，姓名为”赵新”，年龄为20岁

```
INSERT
INTO IS_Student
VALUES(‘201215129’,’赵新’,20);
```

转换为对基本表的更新:

```
INSERT
INTO   Student(Sno,Sname,Sage,Sdept)
VALUES(‘200215129 ','赵新',20,'IS' );
```

更新视图（续）

[例,第6版]删除信息系学生视图IS_Student中学号为”201215129”的记录

```
DELETE
FROM IS_Student
WHERE Sno= ' 201215129 ';
```

转换为对基本表的更新:

```
DELETE
FROM Student
WHERE Sno= ' 201215129 ' AND Sdept= 'IS';
```

更新视图（续）

- 更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新

例：定义的视图S_G为不可更新视图，修改平均成绩。

```
UPDATE  S_G
SET     Gavg=90
WHERE  Sno= '201215121';
```

这个对视图的更新无法转换成对基本表SC的更新

更新视图（续）

- 对视图更新的限制原则(以具体版本特性为准):
  - 若视图由两个以上基本表导出，则此视图不允许更新。
  - 若视图字段来自字段表达式或常数，则不允许对此视图执行INSERT和UPDATE操作，但允许执行DELETE操作。
  - 若视图的字段来自聚集函数或窗口函数，定义中含有GROUP BY，DISTINCT，Limit等短语，则此视图不允许更新。
  - 若视图定义中有嵌套查询，且内层查询的FROM子句中涉及的表也是导出该视图的基本表，则此视图不允许更新。
  - 不允许更新的视图上定义的视图也不允许更新。

修改视图

ALTER VIEW <视图名> AS <SELECT语句>

视图的价值

- 视图能够简化用户的操作(复杂查询)
- 视图使用户能以多种角度看待同一数据
- 视图对重构数据库提供了一定程度的逻辑独立性
- 视图能够对机密数据提供安全保护
- 适当的利用视图可以更清晰的表达查询

第3章 关系数据库存储管理

- 3.1 SQL概述
- 3.2 数据定义
- 3.3 基本SQL查询
- 3.4 数据更新
- 3.5 视图
- 3.6 小结

3.6 小结

- SQL语言是关系数据库管理系统（RDBMS）的标准接口，包括定义数据、查询数据和更新数据，以及视图的使用。
- 本章基于SQL92标准，实际RDBMS在具体实现时可能存在差异，需查阅相应SQL参考手册，OJ/考试以Mysql语法为准。
- 难点在于查询语句，可以结合关系代数理解，并通过上机实验大量练习提高熟练度。

3.6 小结

- DML（data manipulation language）数据操纵语言：

```
SELECT 列名称 FROM 表名称
UPDATE 表名称 SET 列名称 = 新值 WHERE 列名称 = 某值
INSERT INTO table_name (列1, 列2,...) VALUES (值1, 值2,....) DELETE FROM 表名称 WHERE 列名称 = 值
```

DDL（data definition language）数据库定义语言

```
CREATE TABLE 表名称 ( 列名称1 数据类型, 列名称2 数据类型,.... )
ALTER TABLE table_name
ALTER COLUMN column_name datatype
DROP TABLE 表名称
DROP DATABASE 数据库名称
```

select易错点

- 多表需要每张表都有连接字段，否则出现笛卡尔积即结果集排列组合;
- sql语句不能出现多个分号，一条长语句只有一个分号;
- 注意去重， DISTINCT ，可以用于多个字段;
- max聚集函数查的是最大值，并不是最大值对应的记录(因为符合max的可能有多行，也并不是其中一行)
- 聚集函数仅用于select和having，不用于where;
- select 后出现的非聚集函数列，包含(即子集)在group by后出现的列，否则数据误导。
- Having仅用于group by后;
- order by可以按多列依次排序